

LAB6_REPORT_Forwarding & Bp

컴퓨터소프트웨어학부 2021026108 김민제

구현 환경

운영체제 : macOS Sonoma

verilog : iverilog

vcd : gtkwave

c++ : Apple clang version 16.0.0 (clang-1600.0.26.6)

전체 구조 분석 및 설계

HW05에서 구현했던 파이프라인에 FORWARD와 BP 유닛을 추가하여 포워딩을 지원하고 nextPC를 예측하도록 했습니다. Forwarding에서는 hazard가 있다면 ex단계에서 바로 앞 단계의 값을 가져와 계산할 수 있도록 했습니다. 하지만 이때 값을 당겨서 쓰기 때문에 load after use 하는 과정에서 hazard가 남아 1 cycle stall을 해주었습니다. BP의 경우 과제의 명세대로 BTB와 PHT를 업데이트 하도록 했습니다. JR도 JUMP와 함께 처리했습니다.

C++ 설계 및 구현

I. FORWARD.h & FORWARD.cpp

ForwardA, ForwardB를 추가하여 각각 rs, rt의 어떤 값을 forwarding 하여 쓸지 찾을 수 있도록 했습니다.

```
if (EX_MEM.WB.RegWrite && EX_MEM.wr_addr != 0 && EX_MEM.wr_addr == rs_l)
    fwd.ForwardA = 2; // from MEM
else if (MEM_WB.WB.RegWrite && MEM_WB.wr_addr != 0 && MEM_WB.wr_addr == rs_l)
    fwd.ForwardA = 1; // from WB
```

II. HAZARD.h & HAZARD.cpp

이전 과제했던 HAZARD는 FORWARDING을 통해서 제거되어 기존의 hazard detection 로직을 지우고 load-use hazard를 탐지할 수 있도록 했습니다.

```
if (ID_EX_curr.M.MemRead) {
    uint32_t ex_dst = ID_EX_curr.EX.RegDst ? ID_EX_curr.rd : ID_EX_curr.rt;

    if (ex_dst != 0 &&
        ((use_rs && ex_dst == rs_l) || (use_rt && ex_dst == rt_l))) {
        return true;
    }
}
```

III. BP.h & BP.cpp

- 64개의 BTB entry, 256개의 PHT entry를 만들었습니다. 이때 BTB에는 tag, target, valid 정보를 담았습니다. valid에는 초기화되었는지, jump, branch 명령에 대한 target 값을 가지고 있는지를 저장하도록 했습니다.
- init으로 처음 초기화 해주고 predict에서 다음 pc 값을 예측해주고 update에서 BTB와 PHT를 업데이트 해줍니다. init에서 pht를 weakly not taken으로 초기화해주었습니다.
- predict에서 만약 valid가 jump이고 tag가 일치한다면 BTB target을 반환하고 valid가 branch이고 tag가 일치하고 추가로 PHT의 entry가 taken을 가리킨다면 target을 반환하고 아니라면 그냥 PC+4를 반환하도록 했습니다.

```
if(btb[idx].valid == JUMP && btb[idx].tag == tag){
    return btb[idx].target;
```

```

}
if(btb[idx].valid == BRANCH && btb[idx].tag == tag && pht[pht_idx] >= 2){
    return btb[idx].target;
}
return PC + 4;

```

4. update에서는 각 control flow 명령어에 맞게 valid를 설정해주고 target과 tag를 업데이트해줍니다. 추가로 PHT의 entry를 saturation counter에 맞게 변경시켜줍니다.

```

btb[idx].tag = tag;
btb[idx].target = target;

if(is_Jump){
    btb[idx].valid = JUMP;
}else if(is_Branch){
    btb[idx].valid = BRANCH;
    uint32_t &history = pht[pht_idx];
    if (taken) {
        if (history < 3) history++; // 01→10→11
    } else {
        if (history > 0) history--; // 10→01→00
    }
}
}

```

IV. CPU.h & CPU.cpp

1. CPU.h에 앞서 새로 만든 FORWARD와 BP를 추가했습니다.
2. 처음 PC를 모듈을 통해서 업데이트해줍니다. 그리고 예측해온 값을 latch로 넘겨주어 후에 EX 단계에서 taken 여부를 결정할 때 값이 같은지 확인을 위해 사용합니다.
3. EX 단계에서 forward 유닛을 통해 필요한 값을 forwarding을 통해 가져옵니다.

```

uint32_t forwarded_rt;
// forwarding
fwd = forward.Forwarding(ID_EX_curr.rs, ID_EX_curr.rt, EX_MEM_curr, MEM_WB_curr);
switch (fwd.ForwardA) {
    case 0: operand1 = ID_EX_curr.read_data1; break;
    case 1: operand1 = wb_wr_data; break;
    case 2: operand1 = EX_MEM_curr.alu_result; break;
}
switch (fwd.ForwardB) {
    case 0: forwarded_rt = ID_EX_curr.read_data2; break;
    case 1: forwarded_rt = wb_wr_data; break;
    case 2: forwarded_rt = EX_MEM_curr.alu_result; break;
}

```

4. Jr 명령어의 경우에도 forwarding을 적용해줍니다.

```

uint32_t jr_target = ID_EX_curr.read_data1;
if (fwd.ForwardA == 1) jr_target = wb_wr_data;
else if (fwd.ForwardA == 2) jr_target = EX_MEM_curr.alu_result;

```

5. EX 단계에서 만약 control flow 명령이라면 BTB와 PHT를 업데이트 해주고 예측한 PC와 실제로 branch 해야할 PC 값이 다르다면 flush를 해줍니다.

```

EX_MEM_next.branch_taken = actual_taken;
if (ID_EX_curr.EX.Branch || ID_EX_curr.EX.JR || ID_EX_curr.EX.Jump) {
    bp.update(ID_EX_curr.PC - 4, actual_taken, actual_taken?EX_MEM_next.PC:ID_EX_curr.PC,
        ID_EX_curr.EX.Jump || ID_EX_curr.EX.JR, ID_EX_curr.EX.Branch);
}
if(EX_MEM_next.branch_taken && EX_MEM_next.PC != ID_EX_curr.bh_PC){
    printf("flush\n");
    IF_ID_next.reset();
    ID_EX_next.reset();
    PC_next = EX_MEM_next.PC;
}
}

```

6. 위 수정 외의 부분은 HW5와 거의 동일합니다.

CPP 동작 확인

makefile에 새로 추가한 BP.cpp와 FORWARD.cpp를 추가했습니다.

```
SRC = main.cpp CPU.cpp ALU.cpp ... HAZARD.cpp FORWARD.cpp BP.cpp
```

I. log와 MARS 값 비교

iverilog로 컴파일하여 실행시킨 파일의 결과와 MARS의 결과와 비교한 결과 일치하는 것을 확인했습니다.

추가로 flush와 stall이 일어났을 때 출력을 하도록해 이를 원래 pipeline과 비교하였을때 확연하게 줄어든 것을 확인할 수 있었습니다. 특히 테스트케이스1번에선 stall이 아예 일어나지 않았으며 testcase 3,5번에서는 flush가 2번, 3번으로 매우 줄어든 것을 확인할 수 있습니다.

./cpu ../hex/test1	./cpu ../hex/test2	./cpu ../hex/test3	./cpu ../hex/test5	./cpu ../hex/test7
Loading instruction memory... Starting CPU simulation... c Instruction memory overflow RF states after the program execution \$zero 00000000 \$at ffffffa6e4 \$v0 00000000 \$v1 00000000 \$a0 00000000 \$a1 00000000 \$a2 00000000 \$a3 00000000 \$t0 000001db \$t1 00000001 \$t2 ffffffa6e5 \$t3 00000001 \$t4 fffff4a25 \$t5 00000000 \$t6 00000000 \$t7 00000000 \$s0 ffffffff \$s1 fffffba3a \$s2 00000001 \$s3 fffff4221 \$s4 00000000 \$s5 00000001 \$s6 00000000 \$s7 00000000 \$t8 00000001 \$t9 00000596 \$k0 00000000 \$k1 00000000 \$gp 00001800 \$sp 00003ffc \$fp 00000000 \$ra 00000000	Loading instruction memory... Starting CPU simulation... c Instruction memory overflow RF states after the program execution \$zero 00000000 \$at fffff9000 \$v0 00000005 \$v1 00000000 \$a0 00000000 \$a1 00000000 \$a2 00000000 \$a3 00000000 \$t0 fffff8bdf \$t1 fffff9d354 \$t2 fffff8bdf \$t3 fffff8bdf \$t4 ffea7884 \$t5 fffff9f0cb \$t6 00000000 \$t7 00000000 \$s0 fffff8bdf \$s1 fffff8bdf \$s2 fffff9f0cb \$s3 00000000 \$s4 ffea7884 \$s5 00042808 \$s6 00000000 \$s7 00000000 \$t8 00000000 \$t9 00000000 \$k0 00000000 \$k1 00000000 \$gp 00001800 \$sp 00003ffc \$fp 00000000 \$ra 00000000	Loading instruction memory... Starting CPU simulation... c Instruction memory overflow RF states after the program execution \$zero 00000000 \$at 00000000 \$v0 00000028 \$v1 00000000 \$a0 00000000 \$a1 00000000 \$a2 00000000 \$a3 00000000 \$t0 0000030c \$t1 00000028 \$t2 00000028 \$t3 00000000 \$t4 00000000 \$t5 00000000 \$t6 00000000 \$t7 00000000 \$s0 00000000 \$s1 00000000 \$s2 00000000 \$s3 00000000 \$s4 00000000 \$s5 00000000 \$s6 00000000 \$s7 00000000 \$t8 00000000 \$t9 00000000 \$k0 00000000 \$k1 00000000 \$gp 00001800 \$sp 00003ffc \$fp 00000000 \$ra 00000000	Loading instruction memory... Starting CPU simulation... c Instruction memory overflow RF states after the program execution \$zero 00000000 \$at 224d0000 \$v0 00000005 \$v1 00000000 \$a0 00000000 \$a1 00000000 \$a2 00000000 \$a3 00000000 \$t0 ffffffff \$t1 00000000 \$t2 ffffffff \$t3 ffffffff \$t4 00000000 \$t5 00000000 \$t6 00000000 \$t7 00000000 \$s0 e219cc86 \$s1 11bbba2a \$s2 65a4c2ac \$s3 13d0e99e \$s4 61116734 \$s5 fdead08c \$s6 00000000 \$s7 00000000 \$t8 00000000 \$t9 00000000 \$k0 00000000 \$k1 00000000 \$gp 00001800 \$sp 00003ffc \$fp 00000000 \$ra 00000034	Loading instruction memory... Starting CPU simulation... c Instruction memory overflow RF states after the program execution \$zero 00000000 \$at e2810000 \$v0 00000000 \$v1 00000000 \$a0 00000000 \$a1 00000000 \$a2 00000000 \$a3 00000000 \$t0 ffffffff \$t1 00000000 \$t2 ffffffff \$t3 ffffffff \$t4 00000000 \$t5 00000000 \$t6 00000000 \$t7 00000000 \$s0 00000000 \$s1 00000000 \$s2 00000000 \$s3 00000000 \$s4 00000000 \$s5 00000000 \$s6 00000000 \$s7 00000000 \$t8 00000000 \$t9 00000000 \$k0 00000000 \$k1 00000000 \$gp 00001800 \$sp 00003ffc \$fp 00000000 \$ra 00000000

Verilog 설계 및 구현

I. Forward.v

비동기로 cpp과 동일하게 동작하도록 구현했습니다.

```

assign ForwardA =
(ex_mem_regwrite && (ex_mem_wr_addr != 0) && (ex_mem_wr_addr == rs_l)) ? 2'b10 :
(mem_wb_regwrite && (mem_wb_wr_addr != 0) && (mem_wb_wr_addr == rs_l)) ? 2'b01 :
2'b00;

assign ForwardB =
(ex_mem_regwrite && (ex_mem_wr_addr != 0) && (ex_mem_wr_addr == rt_l)) ? 2'b10 :
(mem_wb_regwrite && (mem_wb_wr_addr != 0) && (mem_wb_wr_addr == rt_l)) ? 2'b01 :
2'b00;

```

II. HAZARD.v

비동기로 latch의 값 설정이 끝난 후에 동작하여 다음 명령어를 실행할지 말지 결정하도록 했습니다. 로직은 cpp과 동일합니다.

```

if (EX_MemRead) begin
  if(((use_rs && (ex_dst == rs_l)) || (use_rt && (ex_dst == rt_l)))) begin
    stall = 1'b1;
  end else begin
    stall = 1'b0;
  end
end
else begin
  stall = 1'b0;
end
end

```

III. BP.v

- 비동기로 update를 실행하도록 했습니다.
 - update 시에 valid를 이용해서 어떠한 명령어의 대한 target인지 기억하도록했습니다.
- 비동기로 predict를 실행하여 latch가 세팅된 이후 다음 pc값을 예측하도록 했습니다.

```

always @(*) begin
  if(btb_valid[btb_idx] == `BP_JUMP && btb_tag[btb_idx] == tag)
    predicted_target = btb_target[btb_idx];
  else if (btb_valid[btb_idx] == `BP_BRANCH && btb_tag[btb_idx] == tag && pht[pht_idx] >= 2) begin
    predicted_target = btb_target[btb_idx];
  end else begin
    predicted_target = pc + 4;
  end
end
...
always @(*) begin
  if(Branch || JR || Jump) begin
    btb_tag[ubtb_idx] <= utag;
    if(branch_taken) begin
      btb_target[ubtb_idx] <= update_target1;
    end else begin
      btb_target[ubtb_idx] <= update_target2;
    end
  end

  if(JR || Jump) begin
    btb_valid[ubtb_idx] <= `BP_JUMP;
  end else if(Branch) begin
    btb_valid[ubtb_idx] <= `BP_BRANCH;
  end
end

```

```

        case (branch_taken)
            1'b1: if (pht[upht_idx] < 2'b11) pht[upht_idx] <= pht[upht_idx] + 1;
            1'b0: if (pht[upht_idx] > 2'b00) pht[upht_idx] <= pht[upht_idx] - 1;
        endcase
    end
end
end

```

IV. CPU.v

1. cpp과 동일하게 구현했습니다.
2. BP와 FORWARD 유닛을 추가해주었습니다.
3. operand를 정하는 부분에 forwarding을 하도록 해주었습니다.

```

assign operand1 = (ForwardA == 2'b10) ? EX_MEM_alu_result :
                  (ForwardA == 2'b01) ? wb_wr_data : ID_EX_read_data1;
assign forwarded_rt = (ForwardB == 2'b10) ? EX_MEM_alu_result :
                    (ForwardB == 2'b01) ? wb_wr_data : ID_EX_read_data2;

assign operand2 = (ID_EX_EX_ALUSrc == 1'b1) ? ID_EX_ext_imm : forwarded_rt;

```

4. JR 명령에서도 forwarding 한 값을 사용하도록 했습니다.

```

if (ForwardA == 1)
    EX_MEM_PC <= wb_wr_data;
else if (ForwardA == 2)
    EX_MEM_PC <= EX_MEM_alu_result;
else
    EX_MEM_PC <= ID_EX_read_data1;

```

5. 만약 flush 조건이라면 Flush를 처리하도록 했습니다.

```

wire flush;
assign flush = (branch_taken && EX_MEM_PC != EX_MEM_bh_PC);

```

Verilog 동작 확인

아래 명령어로 컴파일하여 cpu 파일을 실행하여 동작 결과를 확인했습니다.

```

iverilog -o cpu ALU.v CPU.v CPU_tb.v CTRL.v
GLOBAL.v MEM.v RF.v HAZARD.v FORWARD.v BP.v

```

확인 결과 모두 SUCCESS가 뜨며 올바르게 동작하는 것을 확인할 수 있었습니다.

추가로 stall과 flush가 일어날 때 출력을 하도록하여 forwarding과 bp가 올바르게 동작하는지 확인했습니다.

testcase 1번에서는 stall이 걸리지 않으니 Forwarding이 올바르게 동작하는 것을 확인했습니다.

testcase 4번에서는 flush가 지난 과제보다 훨씬 적게 나오는 것을 확인하여 bp가 올바르게 동작하는 것을 확인했습니다. ⇒ cpp과 동일하게 flush가 출력이 되었습니다.

```
> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:58: $finish called at 15365000 (1ps)
```

```
> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
flush
flush
Program Terminate

Simulation success!!!
CPU_tb.v:58: $finish called at 19205000 (1ps)
```

Trouble Shooting

I. Verilog에서 BP update시 target 관련 문제

cpp에서 bp.update에서 Latch의 curr 값과 next 값을 혼용하여 사용해서 verilog에 그대로 옮겼더니 문제가 생겼습니다. curr 과 next값이 동시에 존재할 수 없는데 혼용하여 사용하여 동기적으로 업데이트 되는 것이 불가능했습니다.

```
bp.update(ID_EX_curr.PC - 4, actual_taken, actual_taken?EX_MEM_next.PC:
ID_EX_curr.PC, ID_EX_curr.EX.Jump || ID_EX_curr.EX.JR, ID_EX_curr.EX.Branch);
```

verilog에서도 동기로 구현하려했으나 EX_MEM_next와 actual_taken이 생성되는 시점과 ID_EX 값들이 업데이트 되는 시점이 맞지 않아 전체적인 구조를 바꿔야해서 그냥 비동기로 구현했습니다. 그리고 Update 되는 시점을 EX 명령어가 끝난 직후로 했습니다.